

Our team's primary objective for this phase was to transform the **inventory_sales_system** from a basic functional database into a high-performance system capable of handling complex analytical queries efficiently. We began by establishing a performance baseline to identify where the system was lagging. Mary's initial analysis revealed that several key queries were struggling with significant execution delays. The main culprit was the database performing full table scans, where it had to manually sift through over 28,000 rows in the orders table just to calculate basic monthly totals. This led to the heavy use of "filesorts" and temporary tables, which are notorious for slowing down database performance.

To address these bottlenecks, Francheska implemented a strategic indexing plan. We didn't just add indexes randomly; we specifically targeted the columns that were being used for joining tables and filtering results. We added indexes to `order_date` in the orders table to help with time-based trends, and established indexes on `order_id`, `product_id`, and `customer_id` across our tables. These indexes act like a map for the database, allowing it to jump straight to the relevant data instead of reading every single record from start to finish.

The performance gains we recorded after these changes were massive. For our Query 1, which tracks total sales per month, the execution time dropped from 0.7543 seconds to 0.4331 seconds, representing a 42% improvement in speed. Our best-selling products analysis, Query 2, saw an even more dramatic shift, becoming 61% faster as the time plummeted from 0.8434 seconds to just 0.3247 seconds. Additionally, our customer value calculations in Query 3 were cut by more than half, moving from 0.2530 seconds to a much more efficient 0.1215 seconds.

Finally, Akisha conducted a thorough verification phase to ensure that our optimizations didn't introduce any bugs or errors. She confirmed that all queries were executed successfully and that our table joins and indexes are fully verified with no SQL errors detected. She also noted that our Year-over-Year trend query saw a significant performance gain, reducing its execution time from 0.2875 seconds to 0.1810 seconds. By documenting these specific strategies and the resulting data, we can clearly demonstrate that our group successfully optimized the database for professional-grade performance.